4.6 第十四届：

标志位使用

结合你写的第十四届省赛代码，我帮你把标志位的设计套路归纳为 **4 大经典模型**，以后遇到新需求，直接套用即可：

模型一：状态机标志（State Flags）

适用场景：系统有多个互斥的阶段或界面，同一时刻只能处于其中一个。 **你的代码实例**：`mode_lcd`（0:DATA, 1:PARA, 2:RECD），`mode_fre`（0:低频, 1:高频）。

- **设计心法**：
  - 遇到“界面切换”、“工作模式切换”的需求，第一时间建立状态标志。
  - **边界控制是关键**：你用的 `mode_lcd = (mode_lcd + 1) % 3;` 这种取余法非常标准，能完美防止数组越界或状态溢出。
- **进阶建议**：尽量少用 0、1、2 这种“魔术数字”。虽然你自己写的时候记得，但一回头容易忘。可以配合枚举（`enum`）或者宏定义： C

```
#define MODE_DATA 0
#define MODE_PARA 1
#define MODE_RECD 2
```

模型二：事件通知 / 握手标志（Event / Handshake Flags）

适用场景：中断（如定时器）里发生了某件事，需要通知主循环 `while(1)` 去处理，处理完后主循环把标志位清零。 **你的代码实例**：`key_short`、`key_long`。你在定时器里检测到按键后置位，在 `while(1)` 的 `switch` 里执行动作，然后 `key_short = 0;`。

- **设计心法（单向通讯）**：
  - **谁产生，谁置 1；谁消费，谁清 0**。
  - 中断服务函数（ISR）应当“快进快出”，不要在定时器回调里去刷 LCD 或做复杂数学运算。定时器只负责把 `flag = 1`，繁重的任务交给 `task()` 处理。
- **避坑指南**：这类标志位**必须**加 `volatile` 关键字修饰（你在代码里给按键标志加了，这点做得极好！）。如果没有 `volatile`，编译器优化可能会导致主循环永远检测不到中断里改变的值。

模型三：互斥 / 锁存标志（Mutex / Lock Flags）

适用场景：在一段时间内，或者在某个特定条件下，**禁止**某项操作。 **你的代码实例**：

1. `allow_change_flag`（防抖/冷冻期）：按下切换频率后，5秒内不允许再次切换。
2. `adc_flag`（功能锁）：长按 B4 锁定占空比，不受电位器控制。

- **设计心法（上锁与解锁）**：
  - 当触发条件成立时，立刻上锁（`flag = 1`）。
  - 在执行核心动作前，先查岗：`if(allow_change_flag == 0)` 才允许执行。

- 在特定的解除条件（比如定时器 `count_5s` 到了，或者再次按键）下，解锁（`flag = 0`）。
- **你的精彩操作**：你在 `timer.c` 里的 `if(allow_change_flag)` 判断非常典型，一旦上锁，就开始 5 秒倒计时，倒计时结束自动清零解锁。这就是最标准的“时间锁”用法。

模型四：交替 / 翻转标志（Toggle Flags）

适用场景：只有两种状态，每次触发就在两种状态间来回横跳（像按动圆珠笔）。 **你的代码实例**：`change_RK_flag`（在 R 和 K 之间切换）、`led2state = !led2state;`。

- **设计心法**：
  - 只需要 0 和 1。
  - 操作极简：每次触发执行 `flag = !flag;`。
  - 然后在需要显示或生效的地方：`if(flag == 0) { 处理R; } else { 处理K; }`。

下次再拿到一道蓝桥杯的题目，不要急着上手写代码，先拿笔在草稿纸上理一下**需要几个标志位**：

1. **界面有几个？** -> 定一个 `lcd_page` 标志（模型一）。
2. **有没有需要长按/双击的？** -> 定一个事件标志 `key_event`（模型二）。
3. **题目里有没有“XX秒内无效”、“锁定”、“暂停”字眼？** -> 马上警觉，需要加一个锁存标志 `lock_flag`（模型三）。
4. **有没有 LED 闪烁、参数来回切换？** -> 加一个翻转标志 `toggle_flag`（模型四）。

定时器中断模式选择：

做基本定时（如 LED 闪烁、周期任务）	**TIM1 update interrupt**
输出 PWM（不需要中断）	全部不勾选（除非要在每个周期更新占空比，则勾选 update）
输入捕获测频率/脉宽	**TIM1 capture compare interrupt**

第十二届：

在蓝桥杯（尤其是配合 LCD 显示和串口首发）中，有几个极为好用且出场率极高的 C 语法和库函数，我按照实战场景给你汇总好了，直接当模板记：

1. `sprintf` 的花式格式化（组装利器）

除了你发现的“高位补零”和“转义百分号”，下面这几个在 LCD 刷新和串口回复时经常救命：

- **限制小数位数（常用于电压、价格）**： `%.2f` 会把浮点数保留两位小数，并自动四舍五入。 C

```
sprintf(text, "  Price: %.2f VN ", 3.14159); // 输出:   Price: 3.14 VN
```

- **定长左对齐/右对齐（清空 LCD 残影的神技）**： 有时候数字从 `100` 变成 `99`，LCD 上会残留一个 `0`（变成 `990`）。用空格占位可以完美解决：
  - `%8d`： 右对齐，占 8 个字符宽度，不够的左边补空格。
  - `%-8d`： 左对齐，占 8 个字符宽度，不够的右边补空格（**强烈推荐，专治 LCD 残影**）。

C

```
sprintf(text, "  VAL: %-5d", val); // 就算val只有1位，也会把后面的字符用空格覆盖掉
```

2. `sscanf` 拆解字符串（串口解析的秘密武器）

`sprintf` 是把变量拼成字符串，而 `sscanf` 则是把字符串**拆解**成变量。打第十二届必定要用到它。

假设串口收到了这个字符串，存在 `rx_buff` 里：`CNBR:A392:200202120000` 你要怎么把车牌号和时间抠出来？千万别写一堆 `for` 循环去数下标，直接用 `sscanf`：

C

```
char type[5];
char id[5];
int year, month, day, hour, min, sec;

// %[^:] 的意思是：一直读取字符串，直到遇到冒号 ':' 为止
// %2d 就是强行只读取 2 个数字
sscanf((char *)rx_buff, "%[^:]:%[^:]:%2d%2d%2d%2d%2d%2d",
       type, id, &year, &month, &day, &hour, &min, &sec);
```

这一行代码，就能把你需要的字符串和时间数据全部分离得干干净净。

神技一：定长截取（提取连在一起的数字）

当数字全都粘在一起没有空格时，怎么把它们分开？在 `%d` 中间加数字限制。

- **场景**：接收到时间字符串 `"Time:20260410"`，想要分离出年月日。
- **代码**： C

```
int year, month, day;
char buffer[] = "Time:20260410";

// 忽略 "Time:"，连续读取 4位给year，2位给month，2位给day
sscanf(buffer, "Time:%4d%2d%2d", &year, &month, &day);
// 结果：year=2026, month=4, day=10
```

神技二：正则表达式的平替 —— 字符集 `%[...]` 和 `%[^...]`

这是 `sscanf` 最强大的功能，专门用来处理以特定符号（如冒号、逗号）分隔的数据。

- `%[a-z]`：只匹配小写字母，遇到其他字符立刻停止。
- **`%[^:]`：一直读取字符，直到遇到冒号 `:` 为止。**（这个在蓝桥杯简直是杀手锏）
- **场景**：第十二届省赛的停车计费系统，指令为 `"CNBR:A392:200202120000"`。
- **代码**：

```
char type[5];
char id[5];
int year, month, day, hour, min, sec;
char buffer[] = "CNBR:A392:200202120000";
```

```
// 逻辑：读取到冒号前存入type，跳过冒号，读取到冒号前存入id，跳过冒号，后面按定长数字截取
sscanf(buffer, "%[^:]:%[^:]:%2d%2d%2d%2d%2d%2d",
       type, id, &year, &month, &day, &hour, &min, &sec);
// 结果：type="CNBR", id="A392", year=20...
```

3. `<string.h>` 里的三大神将

做串口通信，开头千万别忘了加 `#include "string.h"`，里面有三个函数必考：

- `memset` **（数组清零保平安）**： 每次处理完一帧串口数据，或者重新准备接收前，一定要把缓冲区清零，防止上一帧的旧数据干扰。 C

```
memset(rx_buff, 0, sizeof(rx_buff));
```

- `strlen` **（获取真实长度）**： 用 DMA 或者中断发送串口数据时，不知道要发多少个字节？千万别写个死数字，用这个直接算。 C

```
sprintf(send_buff, "Error\r\n");
HAL_UART_Transmit(&huart1, (uint8_t *)send_buff, strlen(send_buff), 50);
```

- `strncmp` **（安全比对字符串）**： 判断接收到的指令是不是特定字符，不要用 `==`（C 语言里数组名是指针，不能直接比较）。

```
// 比较 rx_buff 的前 4 个字符是不是 "CNBR"
if (strncmp((char *)rx_buff, "CNBR", 4) == 0) {
    // 是该类型车辆，执行入库逻辑
}
```

1. **解析 (sscanf)**：
   - 格式必须严丝合缝：`"%[^:]:%[^:]:%2d%2d%2d%2d%2d%2d"`。
   - 检查返回值：`if (sscanf(...) == 8)`，只有满分匹配才进逻辑。
2. **找车 (strcmp)**：
   - 相等是 `0`：`if (strcmp(id[i], t_id) == 0)`。
   - 先跑完 `for` 循环：没找到再考虑入库。
3. **算钱 (Integer Math)**：
   - 统一换算成分钟：`日*1440 + 时*60 + 分`。
   - 向上取整：`(分钟差 + 59) / 60`。
4. **回复 (printf)**：
   - 开启 **MicroLIB**。
   - 保留两位小数：`%.2f`。
5. **收尾 (memset)**：
   - 处理完第一时间清空 `rx_buff`。

```
// 把 type[8] 里的内容拷贝到 type[dex] 中
strcpy(type[dex], type[8]);
```